

SIMATS ENGINEERING
ASSESSMENT TOOL 2
Case study

COURSE NAME: OBJECT ORIENTED ANALYSIS AND DESIGN
COURSE CODE: CSA11

COURSE OUTCOME COVERED

CO3: Analyze system requirements using object-oriented techniques to identify classes, attributes, and relationships and develop use case models. (BL4)

Assessment Tool Weightage: 40%

Code of Conduct:

*I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

SIMATS ENGINEERING
ASSESSMENT TOOL 2 Case
study

COURSE NAME: OBJECT ORIENTED ANALYSIS AND DESIGN COURSE CODE:
CSA1108

COURSE OUTCOME COVERED

CO3: Analyze system requirements using object-oriented techniques to identify classes, attributes, and relationships and develop use case models. (BL4)

Assessment Tool Weightage: 40%

Code of Conduct:

I PAVITHRA SRI S(192572079) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Pavithra sri s

Case study

QUESTIONS:

Total Marks:40

1. Online Hotel Reservation System

Introduction

The Online Hotel Reservation System is a web-based application that enables customers to search for hotels, check room availability, book rooms, make secure online payments, and provide reviews after their stay. Hotel managers are responsible for updating room availability, room pricing, and hotel information, while administrators manage users, bookings, payments, and overall system operations. The system improves booking efficiency, reduces manual work, ensures secure transaction processing, and provides customers with a convenient reservation experience.

Application of Object-Oriented Principles 1. Encapsulation

Encapsulation is achieved by keeping the data and the operations related to that data within individual classes. Each class stores its own attributes and provides methods to access or modify them in a controlled manner. For example, the **Customer** class contains customer details such as customer ID, name, email, and phone number, while methods like `register()` and `bookRoom()` manage customer activities. Similarly, the **Booking** class stores booking information and provides methods to create or cancel reservations. This approach improves data security, prevents unauthorized access, and increases code maintainability.

2. Inheritance

Inheritance promotes code reusability by allowing common features to be shared among multiple classes. A general **User** class can contain common attributes such as User ID, Name, Email, Phone Number, and Login Credentials. Specialized classes such as **Customer**, **Hotel Manager**, and **Administrator** inherit these common properties while adding their own unique functions. For example, the Customer can search hotels and book rooms, the Hotel Manager updates room availability and pricing, and the Administrator manages users and transactions.

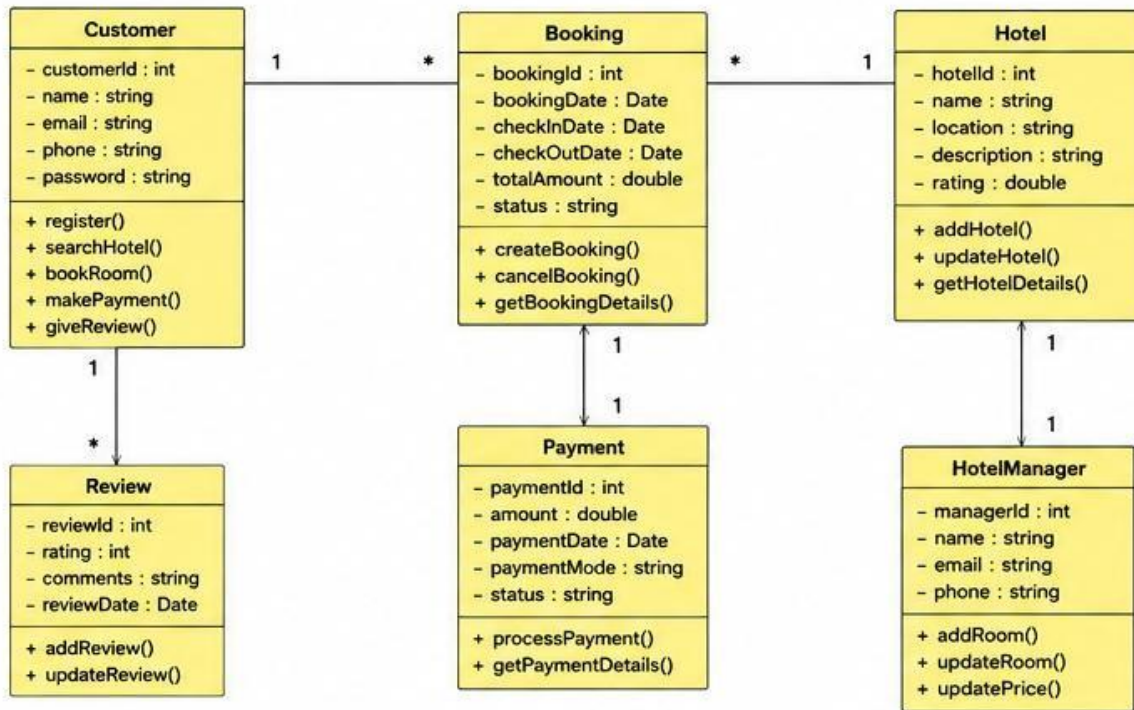
3. Polymorphism

Polymorphism allows different classes to perform similar operations in different ways. For example, the `login()` operation behaves differently depending on whether the user is a Customer, Hotel Manager, or Administrator. Likewise, methods such as `displayDetails()` or `manageBooking()` can provide different

implementations based on the object that invokes them. This improves flexibility and makes the system easier to extend with new user roles in the future.

Major Classes and Their Responsibilities

Class	Responsibility
Customer	Registers, searches hotels, books rooms, makes payments, submits reviews
Booking	Stores booking information, creates and cancels reservations
Hotel	Maintains hotel information, room availability, pricing, and ratings
Hotel Manager	Updates rooms, pricing, and hotel details
Payment	Processes online payments and stores payment details
Review	Stores customer ratings and feedback



Suitable SDLC Model Agile Model

The Agile Software Development Life Cycle (SDLC) model is the most suitable choice for developing the Online Hotel Reservation System because it supports iterative development, continuous testing, and frequent customer feedback. Since hotel booking systems require regular updates such as new payment methods, promotional offers, improved security, and enhanced user interfaces, Agile enables developers to implement these changes quickly without affecting the overall system. Each iteration delivers a functional module that can be tested and improved based on user requirements.

Justification

The Agile model provides flexibility, faster development, and continuous improvement throughout the project lifecycle. It allows developers to respond quickly to changing business requirements and customer expectations. Regular testing ensures that software defects are identified early, reducing development risks and improving software quality. Agile also encourages collaboration among customers, developers, hotel managers, and administrators, resulting in a reliable, secure, and userfriendly hotel reservation system.

Conclusion

The Online Hotel Reservation System effectively applies object-oriented principles such as encapsulation, inheritance, and polymorphism to create a secure, reusable, and maintainable software solution. The identified classes and their relationships ensure proper system organization, while the Agile SDLC model supports continuous development, testing, and enhancement. This combination

results in a scalable, efficient, and customer-friendly hotel reservation platform that satisfies both business and user requirements.

2. University Management System

Introduction

The University Management System is a software application developed to automate various academic and administrative activities within a university. It manages student admissions, course registration, examination scheduling, result processing, and faculty information efficiently. The system provides different access levels for students, faculty members, examination staff, and administrators, ensuring secure and organized management of university operations. This automation improves accuracy, reduces paperwork, enhances communication, and increases overall operational efficiency.

Object-Oriented System Design

The University Management System follows object-oriented principles by representing realworld entities such as students, faculty members, courses, examinations, and administrators as individual classes. Each class contains its own attributes and methods, making the system modular, reusable, maintainable, and scalable. The relationships among these classes help in organizing university operations effectively.

Class	Responsibilities
Student	Registers courses, views timetable, checks examination results
Faculty	Manages courses, enters student marks, views student information
Course	Stores course details and manages course registration
Examination	Conducts examinations and publishes results
Exam Staff	Creates examination schedules and manages examinations
Administrator	Manages users, departments, courses, and generates reports

Relationships Between Classes

- A **Student** can register for multiple **Courses**.
- A **Faculty** member teaches one or more **Courses**.
- **Examination Staff** manage examinations and publish results.
- The **Administrator** manages students, faculty members, departments, and courses.
- Each **Course** belongs to a department and may have multiple enrolled students.

These relationships ensure proper interaction among different components of the university management system.

Figure 2: University Management System – Use Case Diagram

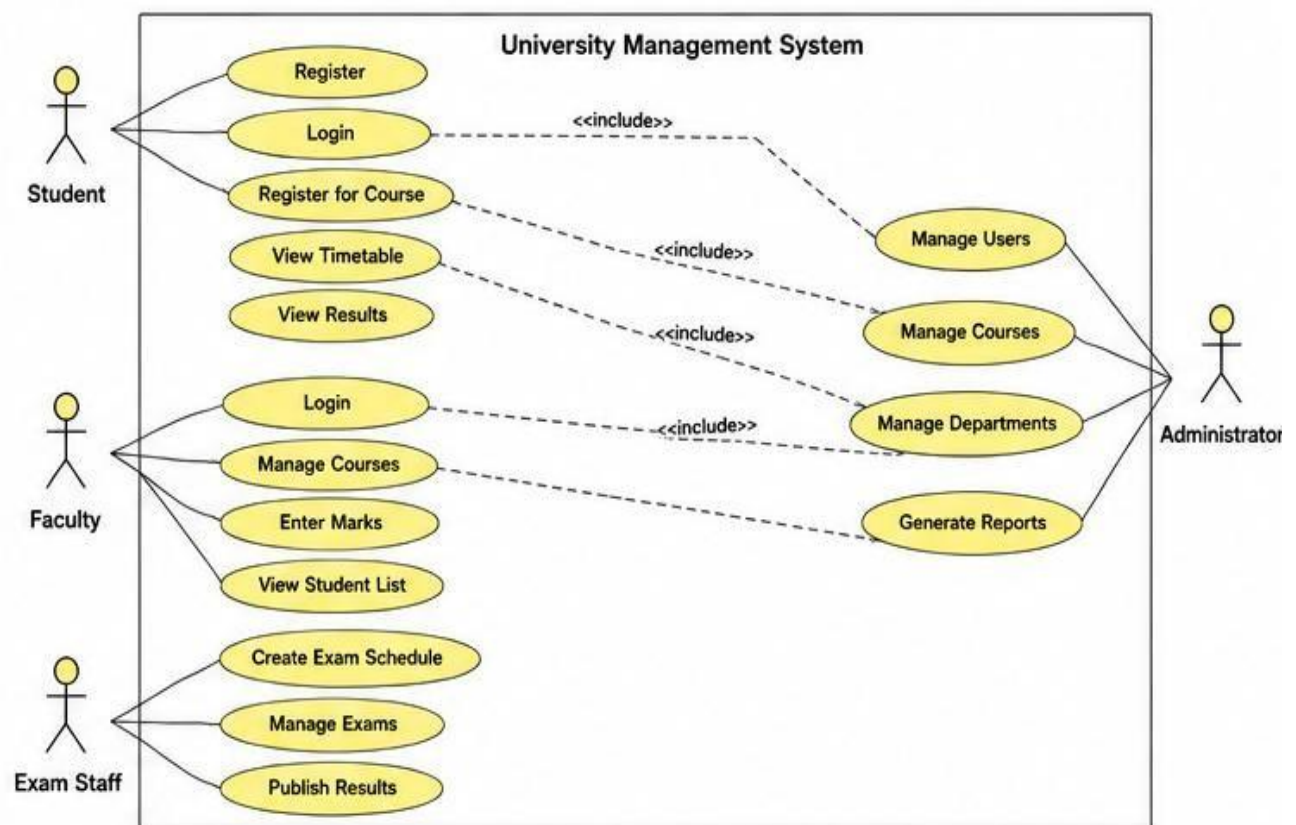


Figure Description:

The Use Case Diagram illustrates the interaction between different actors and the University Management System. It shows how Students, Faculty Members, Examination Staff, and Administrators perform various functions such as course registration, timetable viewing, examination management, result publishing, course management, and report generation. The diagram represents the functional requirements of the system and the interactions between users and the software.

Suitable SDLC Model**Agile Model**

The Agile Software Development Life Cycle (SDLC) model is the most suitable approach for developing the University Management System because university requirements frequently change due to curriculum updates, examination policies, academic regulations, and administrative procedures. Agile allows developers to build the system in small iterations, gather feedback from stakeholders, and continuously improve the software throughout the development process.

Justification

The Agile model provides flexibility, continuous testing, and faster delivery of software modules. New features such as online admissions, attendance management, examination modules, and student portals can be added without affecting the existing system. Frequent communication among developers, faculty members, administrators, and users ensures that the software meets institutional requirements while maintaining high quality and reliability.

Conclusion: The University Management System effectively applies object-oriented concepts by identifying major classes and defining their relationships to represent real-world university operations. The Use Case Diagram clearly models the interactions between users and the system, while the Agile SDLC model supports continuous improvement, adaptability, and efficient software development. This approach results in a reliable, scalable, and userfriendly university management solution.

3. Online Retail Shopping Platform

Introduction

The Online Retail Shopping Platform is an e-commerce application that allows customers to browse products, place orders, make secure online payments, and track deliveries. Vendors can manage product catalogs, update inventory, and monitor orders, while administrators oversee transactions, inventory, and overall system management. The platform improves customer convenience, enhances business efficiency, and provides a secure and reliable shopping experience.

Modular Software Design Using Object-Oriented Techniques

The Online Retail Shopping Platform follows a modular software design by dividing the system into independent classes, where each class performs a specific function. This approach improves maintainability, reusability, scalability, and simplifies future enhancements. The system uses objectoriented concepts such as abstraction, encapsulation, inheritance, and polymorphism to organize different modules efficiently.

The major modules of the system include Customer Management, Product Management, Order Management, Payment Processing, Inventory Management, Vendor Management, and Administration. Each module communicates with others through well-defined relationships while maintaining its own responsibilities.

Major Classes and Responsibilities

Class	Responsibilities
Customer	Browses products, places orders, tracks deliveries, makes payments
Product	Stores product information, price, category, and stock details
Order	Creates customer orders and manages order status
Payment	Processes online payments and records transaction details
Vendor	Manages product catalog and updates inventory
Administrator	Monitors users, transactions, products, and inventory

Advantages of Modular Design

- Each module performs a specific task independently.
- Changes in one module do not affect the entire system.
- Code can be reused in future projects.
- Testing and maintenance become easier.
- New features can be added without redesigning the complete application.
- The system becomes scalable and easier to manage.

Figure 3: Online Retail Shopping Platform – Class Diagram

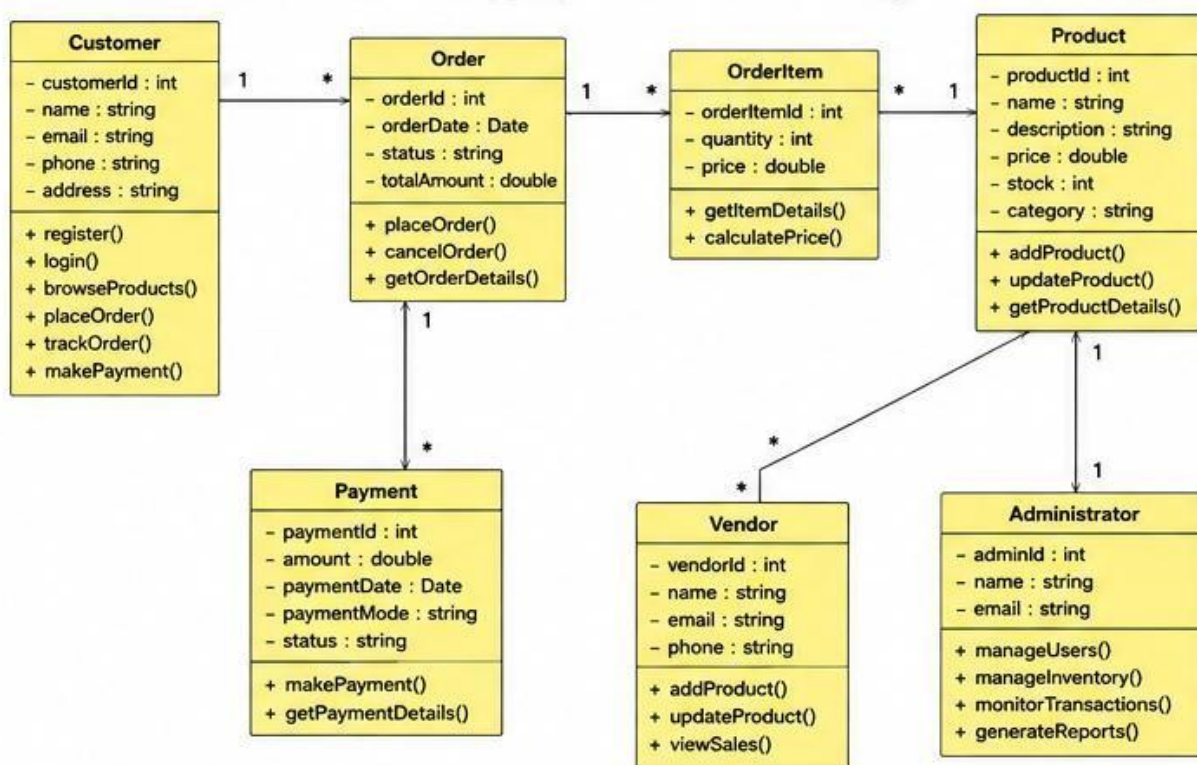


Figure Description:

The Class Diagram illustrates the static structure of the Online Retail Shopping Platform. It shows the major classes such as Customer, Product, Order, Payment, Vendor, and Administrator along with their attributes, methods, and relationships. The diagram demonstrates how object-oriented design supports modular software development by separating different functionalities into independent but interconnected classes.

Evaluation of SDLC Models Waterfall Model

- Suitable for projects with fixed requirements.
- Difficult to accommodate requirement changes during development.

Spiral Model

- Suitable for high-risk projects.
- Includes continuous risk analysis but requires more time and higher development costs.

Agile Model

- Supports iterative development and continuous customer feedback.
- Allows quick implementation of new features and system improvements.
- Facilitates regular testing and faster software delivery.

Recommended SDLC Model Agile Model

The Agile SDLC model is the most appropriate choice for developing the Online Retail Shopping Platform because customer requirements, payment technologies, security standards, and product catalogs change frequently. Agile enables developers to release software in iterations, gather customer feedback, and continuously improve the platform. It also supports rapid bug fixing, frequent updates, and integration of new business features.

Justification

The Agile model provides flexibility, faster delivery, continuous testing, and effective collaboration among developers, vendors, administrators, and customers. It ensures that new payment methods, promotional offers, inventory updates, and user interface improvements can be implemented efficiently without affecting existing system functionality. This results in a secure, scalable, and user-friendly online shopping platform.

Conclusion

The Online Retail Shopping Platform achieves modular software design through object-oriented techniques by organizing the system into independent classes with clearly defined responsibilities. The Class Diagram effectively represents the system structure, while the Agile SDLC model ensures continuous improvement, adaptability, and efficient software development. This approach delivers a reliable, maintainable, and scalable e-commerce solution.

4. Smart Waste Management System

Introduction

The Smart Waste Management System is an IoT-based application designed to improve the efficiency of waste collection and disposal. Smart bins equipped with sensors continuously monitor waste levels and automatically notify collection vehicles when the bins are full. Citizens can report waste-related issues through a mobile application, while administrators monitor the overall waste management process. This system helps reduce manual effort, minimizes overflowing bins, optimizes collection routes, and promotes a cleaner and healthier environment.

Application of Object-Oriented System Development Principles

The Smart Waste Management System applies object-oriented principles by representing each real-world entity as a separate class. Every class contains its own attributes and methods, making the system modular, reusable, and easy to maintain. The interaction between these classes enables efficient communication among sensors, collection vehicles, citizens, and administrators.

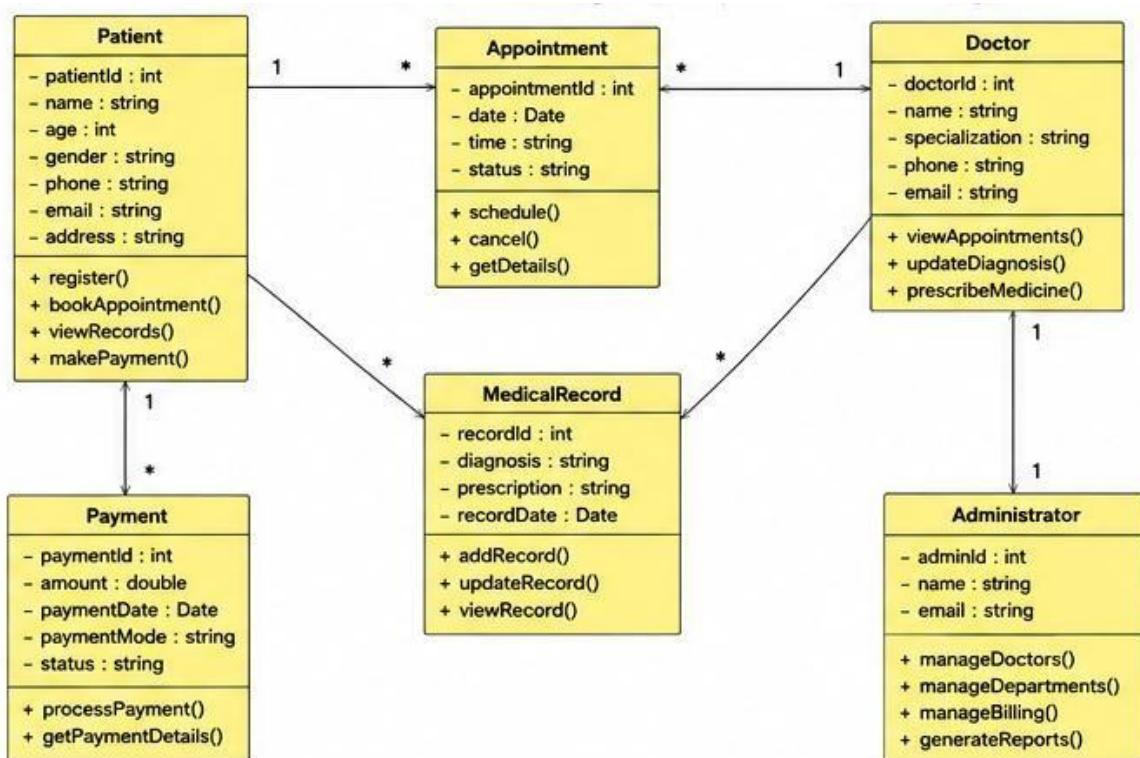
Major Classes and Responsibilities

Class	Responsibilities
Smart Bin	Monitors waste level and sends notifications when full
Sensor	Detects and updates the waste level of each bin
Collection Vehicle	Receives alerts and collects waste from full bins
Citizen	Reports waste-related issues through the mobile application
Administrator	Monitors the system, manages reports, and oversees operations
Waste Report	Stores complaint details and report status

Relationships Between Classes

- Each **Smart Bin** is connected to a **Sensor** that monitors its waste level.
- A **Sensor** sends notifications to the **Collection Vehicle** when the bin becomes full.
- A **Citizen** can submit one or more **Waste Reports** through the mobile application.
- The **Administrator** manages Smart Bins, Citizens, Collection Vehicles, and Waste Reports.
- The **Collection Vehicle** updates the collection status after clearing the waste. These relationships ensure efficient communication and coordination among all components of the system.

Figure 4: Smart Waste Management System – Use Case Diagram



Figure

Description:

The Use Case Diagram illustrates the interactions between the actors and the Smart Waste Management System. It shows how Citizens report waste issues, Sensors monitor bin levels, Collection Vehicles receive collection notifications, and Administrators monitor and manage the overall waste management process. The diagram represents the functional requirements of the system and the interaction between users and system components.

Suitable SDLC Model Spiral Model

The **Spiral SDLC Model** is the most suitable choice for developing the Smart Waste Management System because it involves IoT devices, real-time sensor integration, and evolving system requirements. The Spiral Model combines iterative development with continuous risk analysis, making it ideal for systems that require reliable communication between hardware and software components.

Justification

The Spiral Model allows developers to identify and reduce risks during every phase of development. Since the system depends on sensor accuracy, network communication, real-time data processing, and mobile application integration, continuous testing and evaluation are essential. The model also supports gradual system improvements as new technologies and user requirements emerge. This results in a reliable, scalable, and efficient smart waste management solution.

Conclusion

The Smart Waste Management System effectively applies object-oriented principles by organizing the system into independent classes with clearly defined responsibilities. The Use Case Diagram demonstrates how different users and system components interact to perform waste management operations. The Spiral SDLC Model provides a structured and risk-driven approach for developing IoT-based applications, ensuring system reliability, flexibility, and continuous improvement.

RUBRICS FOR CALCULATION:**Case study**

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly
Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	10	Clear, wellstructured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand